

Zig-zag Sort

We learned in class that comparison-based sorting has a lower bound of $\Theta(n \log n)$. This comes from the fact that there are $n!$ possible orderings of n elements, and a comparison only gives two possible outcomes. Because of this, a comparison-based algorithm needs enough comparisons to distinguish between all of those possibilities. Algorithms like Merge Sort and Heap Sort already reach $O(n \log(n))$, so at first it might seem like there would not be much left to improve with sorting. However, one area that researchers still study is what happens when we add extra restrictions to the sorting problem. An example of this is Zig-zag Sort, which was introduced by Michael T. Goodrich in 2014. Zig-zag Sort is a deterministic, data-oblivious sorting algorithm that runs in $O(n \log(n))$ time.

The problem Zig-zag Sort is trying to solve

Most sorting algorithms we learned in class make decisions based on the actual values in the array. For example, during Merge Sort, the algorithm compares the next elements from two sorted lists:

Left: [2, 7]

Right: [1, 9]

It compares 2 and 1. Since 1 is smaller, it takes 1 from the right list. The result of that comparison affects what the algorithm looks at next.

Quicksort has the same general issue. It compares values against a pivot and decides where to move them based on whether they are smaller or larger than that pivot.

Zig-zag Sort works under an extra restriction called data obliviousness. The positions that the algorithm compares are determined ahead of time. They do not change based on the values in the input.

For example, an oblivious algorithm could have a pattern like:

Compare positions 1 and 5

Compare positions 2 and 7

Compare positions 3 and 8

Compare positions 1 and 2

That pattern would be the same whether the input was:

1 2 3 4 5 6 7 8

or:

8 7 6 5 4 3 2 1

The values can determine whether two elements are swapped, but they do not determine which positions get compared next.

This property is useful in situations where someone might be able to observe which parts of memory an algorithm accesses. Even if the actual values are protected, a value-dependent access pattern could reveal information about the data. Goodrich connects data-oblivious sorting to areas such as oblivious RAM and privacy-preserving cloud computation.

How Zig-zag Sort works

The main idea behind Zig-zag Sort is to repeatedly divide the array into smaller blocks. At each stage, the algorithm tries to get more of the small values toward the left and more of the large values toward the right.

For example, the array starts as one large block:

[A]

It is then divided:

[A1][A2]

Then divided again:

[A1][A2][A3][A4]

This continues until the blocks eventually contain individual elements.

A major tool used during this process is called an epsilon-halver. A halver takes two equal-sized groups and performs a predetermined set of comparisons between them.

The goal is not to completely sort both groups. Instead, the halver makes it so that most of the smaller values end up in the left group and most of the larger values end up in the right group.

Example:

Before:

Left Right
[mixed values] [mixed values]

After a halver:

Left Right
[mostly small] [mostly large]

This was one of the parts of the algorithm that I found most interesting. Merge Sort completely sorts its smaller sections before combining them. Zig-zag Sort can make progress without completely sorting each block. It keeps reducing the number of elements that are badly positioned. Goodrich refers to these misplaced elements as the dirtiness of a block.

The zig and the zag

The name comes from the direction the algorithm moves through the blocks.

First, it performs a left-to-right pass called the zig:

A1 ↔ A2
A2 ↔ A3
A3 ↔ A4

It then performs a right-to-left pass called the zag:

A3 ↔ A4
A2 ↔ A3
A1 ↔ A2

During these passes, neighboring blocks are exchanged and a procedure called Reduce is applied. Reduce uses halvers to decrease the number of misplaced elements. Another way to think about the two passes is that the zig helps push large values that are too far left toward the right side, while the zag helps push small values that are too far right toward the left side. The process then repeats using smaller blocks.

The proof in the paper spends a lot of time talking about arrays containing only 0s and 1s.

For example:

1 0 1 0 0 1 1 0

should become:

0 0 0 0 1 1 1 1

In this example, a 1 that is far to the left can be treated as a misplaced element. A 0 that is too far to the right is also misplaced. This makes it easier to keep track of how much dirtiness remains after each step. This works because of the 0-1 principle for sorting networks. If a fixed comparison network correctly sorts every possible sequence of zeros and ones, then that network will also correctly sort arbitrary comparable values. So proving that Zig-zag Sort works for zeros and ones is enough to prove the more general sorting result.

Running time

Zig-zag Sort still reaches the same asymptotic runtime that we would want from a good comparison sort.

At each level, the work done across all of the blocks is $O(n)$.

The block size is repeatedly cut in half:

1 block

2 blocks

4 blocks

8 blocks

It takes about $\log_2(n)$ levels before the blocks have size one.

Therefore, the total work is $O(n) \cdot O(\log(n)) = O(n \log(n))$.

This matches the lower bound for comparison-based sorting.

What was different?

Data-oblivious sorting existed before Zig-zag Sort. For example, Bitonic Sort follows a fixed pattern of comparisons, but it uses $O(n \log^2(n))$ comparisons. Goodrich's result gave a different construction with a simpler overall structure based on splitting blocks, using halvers, and making the zig and zag passes. The paper states that its constructive versions have constant factors that are orders of magnitude smaller than constructive versions of AKS based on similar halver techniques.

How I learned about it

I learned about Zig-zag Sort mainly from Michael T. Goodrich's 2014 paper, Zig-zag Sort: A Simple Deterministic Data-Oblivious Sorting Algorithm Running in $O(n \log(n))$ Time. I first focused on the introduction and the high-level description of the algorithm instead of trying to understand the entire proof. The main thing I took away from the paper is that reaching $O(n \log(n))$ does not necessarily mean the sorting problem is finished. Adding a requirement such as hiding information about the input changes what kinds of algorithms can be used and creates a different version of the problem.